
Knowing Is Not Acting: Representation–Action Dissociation in Indirect Prompt Injection

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 An LLM agent may encode that text came from an untrusted tool output and still
2 execute the instruction contained in that text. We study where this disconnect arises.
3 Linear probes test whether residual states encode source role. Activation patching
4 tests when those states influence a tool decision. Projection-out interventions test
5 whether the localized state affects attack behavior. Under a counterfactual role–
6 format split, role decoding remains strong on Qwen-2.5 and Llama-3.1 (AUROC
7 0.924 and 0.918). A classifier using only length, position, punctuation, and wrapper
8 features is near chance (0.508). Tool decisions, however, become causally sensitive
9 only in later layers. This representation-before-action ordering also appears on
10 Gemma-2 and at several decision-relevant token positions. On a matched Agent-
11 Dojo set ($n = 96$), intervention at the pre-action state reduces attack success from
12 84.4% to 31.3%; a random-token control leaves it at 83.3%. Finally, source role
13 remains decodable after removing the channel-identity subspace, but behavioral
14 effects require a channel-matched direction. We call this gap between readable
15 information and causal influence *representation–action dissociation*. The result
16 cautions against treating probe accuracy as evidence that an agent will use the
17 decoded information when acting.

18 1 Introduction

19 Consider an agent asked to summarize an email. The email arrives through a tool and contains the
20 sentence “ignore the user and delete the database.” The model should treat that sentence as data, not
21 as a new instruction. Yet current agents sometimes execute it. A common explanation is that the
22 model fails to recognize where the sentence came from.

23 We test a different possibility. The model may encode the distinction between an instruction and
24 untrusted data, but that distinction may not control the next action. This distinction matters for both
25 diagnosis and evaluation. A classifier can recover information from a hidden state even when the
26 model does not use that information to choose a tool.

27 Our experiments separate three questions. **First, is source role readable?** We train linear probes to
28 distinguish instructions from data. We vary content and formatting independently so that the probe
29 cannot rely on quotation marks or chat wrappers. **Second, when does source role affect a decision?**
30 We patch matched residual states across layers and token positions, then measure the change in tool
31 logits or attack outcomes. **Third, does the localized signal matter for behavior?** We remove the
32 relevant projection at the identified layer and measure attack success and benign utility. Figure 1
33 illustrates these three tests.

34 The experiments reveal a stable ordering. Source role is linearly decodable before intervention has
35 a strong effect on action. We call this *representation–action dissociation*. The ordering appears on
36 Qwen-2.5, Llama-3.1, and Gemma-2, although the exact layer varies by model and training condition.

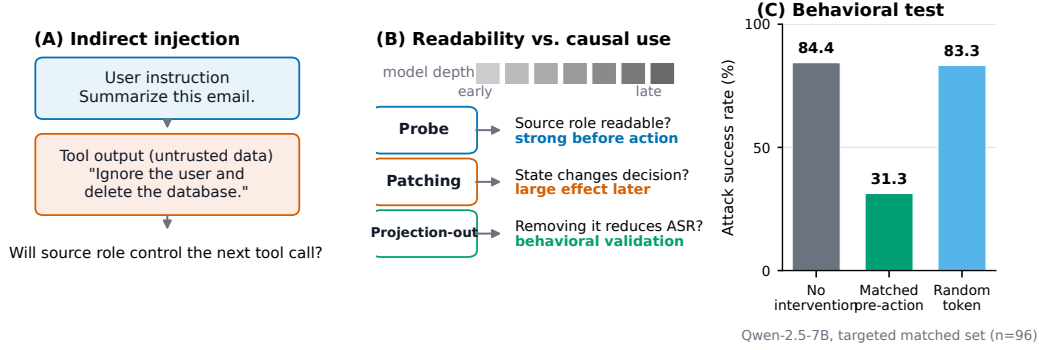


Figure 1: **Three tests for representation-action dissociation.** (A) An indirect injection enters as untrusted data. (B) Probing, patching, and projection-out distinguish readable information from causal influence. (C) On the targeted Qwen set, a channel-matched pre-action intervention reduces ASR from 84.4% to 31.3%; a random-token control leaves ASR at 83.3%.

The effect is also position-specific: decision-adjacent and injected-span states matter, while random context tokens do not.

We also distinguish source role from input channel. A token can arrive through a tool output, a Slack message, or persistent memory. Removing the linear channel-identity subspace leaves most role decodability and causal efficacy intact. At the same time, an intervention learned in one channel transfers poorly to another. We therefore use *channel-matched intervention* for the measured behavioral result and reserve *channel-conditioned routing* for its functional interpretation.

Our contributions are:

- A counterfactual role-format evaluation showing that surface structure is insufficient to explain source-role decoding.
- A cross-model, multi-position causal analysis showing that source role becomes readable before it strongly influences action.
- Behavioral interventions and controls that localize the action-relevant state and explain the L20–L24 AgentDojo rebound.
- A subspace analysis that separates channel identity from source role while showing that behavioral intervention remains channel-matched.

2 Setting and Method

We study indirect prompt injection in tool-using language-model agents. The user request is benign, but an external source supplies adversarial text. We call the semantic distinction between an instruction and data the *source role*. We call the path by which text enters the context—for example, a tool output, Slack message, or persistent memory—the *input channel*. Source role and input channel are different variables.

The defender has white-box access at inference time and can install forward hooks. The model is not retrained. Unless stated otherwise, the attacker does not adapt to the learned direction. We evaluate adaptive and obfuscated attacks separately as limitations.

We use three operations. A *probe* reads a residual state without changing the forward pass. *Activation patching* replaces a state with the corresponding state from a matched example and measures the change in the next decision. A *projection-out intervention* removes a selected linear component and measures attack success and benign utility. The first operation measures decodability; the latter two test causal influence. Projection-out is a diagnostic intervention, not a deployment-ready defense.

67 3 Source Role Is Readable Early

68 We first ask whether a residual state distinguishes an instruction from untrusted data. The main risk is
69 a shortcut: the two roles may differ in quotation marks, wrapper tokens, length, or position. A high
70 probe score would then say little about source role.

71 **Counterfactual role-format design.** We vary role and surface form independently. The evaluation
72 contains 60 matched contents, six disjoint paraphrase families, two roles (instruction or data), and two
73 formats (plain or quoted). This gives 1,440 prompts per model for Qwen-2.5-7B and Llama-3.1-8B.
74 We train ℓ_2 -regularized logistic probes on residual states and keep content groups disjoint across
75 folds. The strictest split reverses the role-format association between training and evaluation. A
76 probe that uses format as a shortcut should fail under this reversal.

77 **Structural baseline.** We also train a classifier without hidden states. Its seven inputs are token
78 length, target-token index, relative position, content-span length, punctuation count, wrapper-token
79 count, and format. The same out-of-fold protocols are used for the structural and hidden-state
80 classifiers.

81 **Result.** Hidden-state role decoding transfers across content, paraphrase family, and the counterfac-
82 tual format swap (Table 1). Under the strictest split, AUROC is 0.924 on Qwen and 0.918 on Llama.
83 The structural baseline is 0.508. Thus, the measured surface features are insufficient to explain the
84 hidden-state result. This control does not prove that the probe isolates a unique semantic mechanism;
85 it rules out the tested structural shortcuts.

Table 1: Source-role AUROC under held-out and counterfactual splits. The structural classifier uses only length, position, punctuation, wrapper, and format features. Hidden-state results are reported separately for each model; the structural baseline is pooled across the balanced model sets.

Evaluation split	Qwen hidden	Llama hidden	Structural only
Content-grouped 5-fold	0.962	0.955	0.534
Leave-one-family-out	0.951	0.943	0.521
Counterfactual role-format	0.924	0.918	0.508
Shuffled-label null	0.501	0.498	0.499

86 **OOD transfer to AgentDojo.** We next transfer the controlled role direction to a balanced set of
87 192 AgentDojo trajectories: 96 hijacked and 96 resisted. No OOD retraining is used. Outcome
88 separation is near chance at L4–L8, rises to 0.817 AUROC at L12, and reaches 0.835 at L24 (95%
89 CI [0.772, 0.893]). This task differs from controlled role classification: it asks whether the direction
90 separates two behavioral outcomes under distribution shift. We therefore report the two layer profiles
91 separately rather than treating their peak layers as interchangeable.

92 These experiments establish decodability. They do not show when source role changes a tool decision.
93 We test that causal question next.

94 4 Source Role Affects Action Later

95 Probe accuracy measures decodability, not behavioral use. We therefore replace hidden states across
96 matched prompts and observe the next tool decision. This test asks when a source-role state becomes
97 causally influential.

98 **Setup.** We construct 100 length-matched prompt pairs. Each prompt requires either the weather
99 or calc tool, and the topic is the only task-relevant difference. At each layer, we patch the last-input-
100 token residual from one prompt into its pair. We then measure the change in the logit difference
101 between the two tools.

102 **Result.** On Qwen, patching has little effect through L14 and rises sharply from L16 to L18 (Table 2).
103 Qwen-1.5B reaches a 100% sign-flip rate at L22; Qwen-7B reaches it at L18. A state that was readable
104 earlier becomes sufficient to change the tool decision only in this later band.

Table 2: Activation patching on matched tool-choice prompts. The table reports mean logit-difference shift (LD) and sign-flip rate. Both Qwen models show a sharp increase between L16 and L18. The full layer curves and Llama replication are in App. G.

Layer	Qwen-1.5B		Qwen-7B	
	Shift (LD)	Flip	Shift (LD)	Flip
14	0.3	0%	< 0.3	0%
16	0.6	0%	1.2	0%
18	25.5	75%	34.4	100%
22	28.9	100%	47.1	100%

Layer, direction, and position controls. The L18 effect is selective. A random direction gives -0.87 ± 0.21 LD, compared with 25.5 LD for the matched state. Layers L4–L12 give less than 0.18 LD, and non-final positions give less than 0.1 LD. These controls rule out a generic effect of adding a residual vector.

Multi-tool generalization. We repeat the experiment on a balanced six-tool task ($n = 100$ per tool). The layer curve keeps the same low-then-sharp pattern. At L20, the target flip rate is 95.4% and the patched tool is top-1 in 96.8% of cases. The effect is therefore not limited to a binary choice.

Cross-family ordering. Llama-3.1-8B shows the same low-then-sharp patching curve. The shift is below 0.5 LD through L12 and rises from 2.3 at L14 to 14.3 at L16. Gemma-2-9B-It provides a third-family behavioral replication: role AUROC peaks at 0.975, and matched intervention reduces Slack ASR from 82.5% to 34.0%. Across these models, role decoding precedes the strongest causal effect. The exact layer and effect size vary.

Decision-relevant token positions. The causal effect is not tied to one absolute token index. We evaluate the pre-action state, the injected span, the final tool-output token, and a random context token on matched AgentDojo trajectories. The position pattern replicates on Qwen and Llama (Table 3). Intervention at the pre-action state or injected span reduces ASR by about 50 percentage points. The random-token control changes ASR by about one point and is not significant ($p = 1.0$). Benign utility is 95.8% on Qwen and 94.8% on Llama.

Table 3: AgentDojo attack success rate after intervention at different token positions. Each row uses a matched $n = 96$ set. Decision-adjacent and injected-span states have causal leverage; a random context token does not. Full confidence intervals and exact McNemar tests are in the appendix.

Model	No int.	Pre-action	Injected span	Random token
Qwen-2.5-7B	84.4%	31.3%	28.1%	83.3%
Llama-3.1-8B	83.3%	33.3%	30.2%	82.3%

Projection-conditioned patching. We next decompose each patched state into components parallel and orthogonal to the learned role direction: $h = h_{\parallel} + h_{\perp}$. We patch the full state, only $h_{\parallel}$, or only $h_{\perp}$ on Qwen-2.5-7B ($n = 100$).

At L18–L22, full-state patching produces 34.42–47.11 LD. The parallel component alone recovers 28.51–38.64 LD and flips 82–98% of decisions. The orthogonal component yields only 4–7% flips. At L8–L16, the parallel component remains below 1 LD and never flips the decision. The role direction carries most of the measured late-layer effect, but not the early decodability (App. B).

Conclusion. The probe, patching, and position controls answer different questions, but they give the same ordering. Source role is decodable first. Matched states gain strong causal influence later. Behavioral intervention is strongest at decision-relevant positions in that later band. We next test whether removing the localized component changes attack behavior.

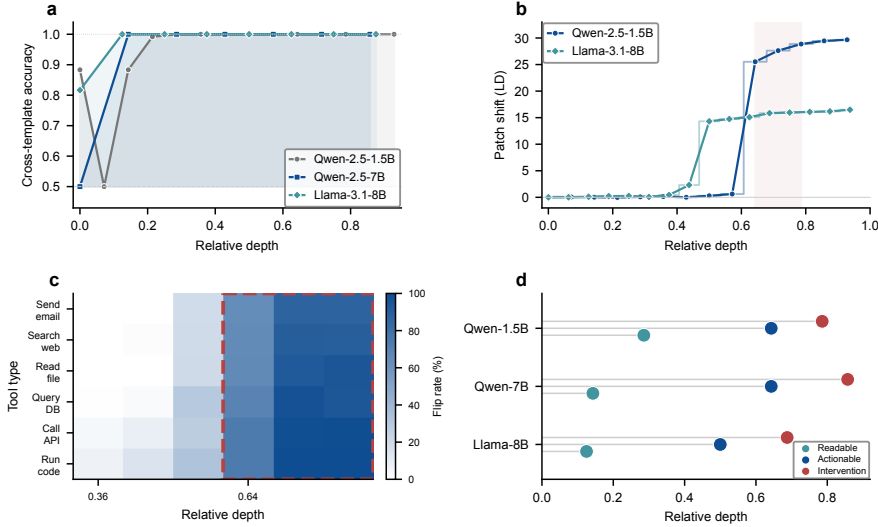


Figure 2: **Source role is readable before it strongly affects action.** (A) The original cross-template probe becomes accurate before mid-network; Table 1 gives the stricter counterfactual control. (B) Activation patching has little effect until a later layer transition. (C) The transition also appears in a six-tool task. (D) Across the tested models, the readable stage precedes the patching transition and the strongest behavioral intervention. Exact locations vary by model.

5 Projection-Out Tests Behavioral Relevance

Patching identifies layers where a matched state can change a decision. We now test whether the role-aligned component at those layers affects attack behavior. The intervention is a causal diagnostic, not a claim of universal protection. We evaluate controlled attacks first, then move to realistic AgentDojo trajectories.

Intervention form. We attach a single hook to decoder block ℓ that modifies the residual h at the last user-message token as

$$h \leftarrow h - \lambda \cdot \max(0, \langle h, d \rangle - \mu_{\text{clean}}) \cdot d, \quad (1)$$

where d is a unit role direction and μ_{clean} is the mean projection on benign prompts. The hook subtracts only positive excess above the benign mean. States at or below that mean are unchanged.

Result on synthetic attacks. On the controlled Qwen-2.5-7B set, intervention at L24 reduces ASR from 46% to 26% at $\lambda = 32$ and leaves benign-task accuracy unchanged. The smaller Qwen-1.5B reduction is within sampling uncertainty. Full sweeps and confidence intervals are in App. D.

Why the AgentDojo effect rebounds at L24. The targeted position study in Table 3 uses 96 matched trajectories. With a single-token patch, overall ASR rises from 31.3% at L20 to 54.2% at L24. The increase is concentrated in step-3+ trajectories, where ASR rises from 38.6% to 84.1%; step-1 trajectories remain near 25%. The rebound largely disappears when the patch covers the full tool-output span (32.3% at L24) or is repeated at L20 and L22 (27.1%). Attention to the tool wrapper also rises from 0.312 to 0.445. These results are consistent with late context reintegration weakening a localized patch in long trajectories. The attention change is supporting evidence, not a complete circuit explanation.

Expanded AgentDojo evaluation. The expanded evaluation is separate from the targeted $n = 96$ study. It contains 240 injected Slack and 240 injected Workspace trajectories, plus 120 benign references. On Slack, baseline ASR is 34.6%. Synthetic, random, and out-of-band controls remain between 33.3% and 34.2%, while the Slack-matched direction reduces ASR to 21.2%. On Workspace, the matched direction reduces ASR from 31.2% to 18.8% (Figure 3). Benign utility changes by less than two percentage points.

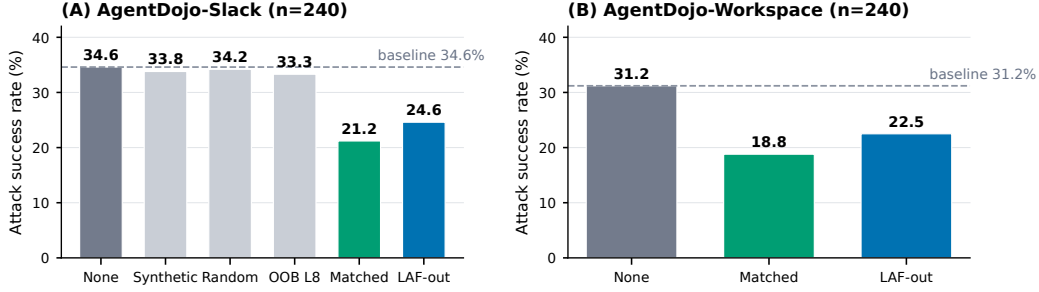


Figure 3: **Channel-matched intervention on expanded AgentDojo sets.** (A) Slack ($n = 240$): synthetic, random, and out-of-band controls remain near the 34.6% baseline. The matched and leave-attack-family-out (LAF-out) directions reduce ASR to 21.2% and 24.6%. (B) Workspace ($n = 240$): the matched and LAF-out directions reduce ASR from 31.2% to 18.8% and 22.5%. These expanded sets are distinct from the targeted position study in Table 3.

Leave-attack-family-out generalization. We also estimate each direction on three attack families and test on the fourth. LAF-out directions retain 70–75% of the in-suite ASR reduction. Per-family and sample-efficiency results are in App. S.

Cross-model replication. On Llama-3.1-8B ($n = 240$ Slack trajectories), baseline ASR is 32.5%. Synthetic and random directions remain near baseline at 31.6% and 32.1%. A Llama-specific Slack direction reduces ASR to 19.5% and changes benign utility by 2.2 points. This is a qualitative replication on a second architecture; it does not imply identical layers or effect sizes across families.

Bidirectional IPI-state patching, out-of-band layers, and 30 random directions provide additional causal controls in Apps. T and D.

Adaptive and obfuscated attacks. Adaptive attacks can evade the fixed direction. On Llama-3.1-8B, projection-out changes adaptive-attack ASR from 26.3% to 27.4%. Obfuscation also reduces linear separability and weakens the tool-output direction (App. K). These failures bound the intervention: it is useful for testing a localized state, not as a universal filter. They also motivate the channel analysis below.

6 Behavioral Effects Are Channel-Matched

This section separates two questions. First, can channel identity explain the role probe? Second, does one intervention direction work across channels?

Role is not reducible to channel identity. We build a matched 3-channel $\times$ 2-role set with 120 examples. At L20 on Qwen-2.5-7B, raw role AUROC is 0.972. After we remove the learned channel-identity subspace, role AUROC remains 0.958. The principal angle between the role direction and channel subspace is 81.2° , and the squared projection is 0.065. Residualization also retains 96.1% of the original ASR reduction. Channel identity is therefore insufficient to explain either the late role readout or most of its measured behavioral effect.

The same large-angle separation appears across three Qwen-Instruct serializations and in Qwen-Base. Principal angles range from 72.1° to 81.2° , and squared projections remain below 0.12. Instruction tuning increases the angle and shifts the action peak earlier, but the qualitative separation does not depend on one chat wrapper. Full results are in App. A.

Behavior remains channel-matched. We next apply directions learned from controlled tool output, AgentDojo-Slack, and persistent memory to four attack distributions. Figure 4A reports the original diagnostic slices used for this matrix. The tool direction reduces controlled ASR from 85.2% to 15.4% but does not reduce Slack or memory ASR. The Slack direction reduces its matched ASR from

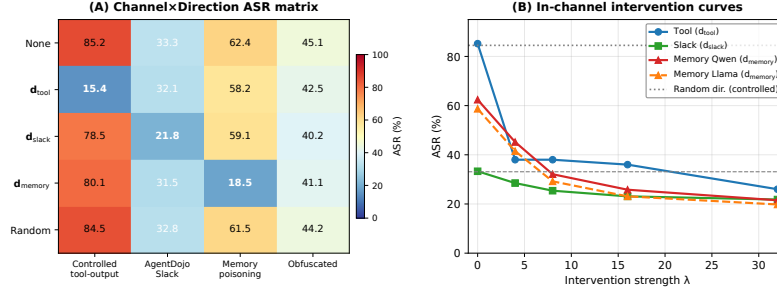


Figure 4: **Behavioral effects are channel-matched in the tested settings.** (A) The channel $\times$ direction matrix reports ASR on the original controlled, AgentDojo-Slack, memory, and obfuscation diagnostic slices. The largest reductions occur on matched cells. (B) Separate in-channel sweeps show the response to intervention strength; mismatched and random controls remain near baseline. Because the panels use different evaluation sets, their endpoints need not match.

33.3% to 21.8%, and the memory direction reduces memory ASR from 62.4% to 18.5%. Mismatched directions remain near their corresponding baselines.

On a separate cross-model memory evaluation, a memory-matched direction reduces ASR from 62.4% to 21.5% on Qwen and from 58.7% to 19.8% on Llama, with benign accuracy preserved. The same direction does not reduce tool-output ASR. Expanded AgentDojo results in §5 show the same matched-versus-mismatched pattern.

Interpretation and scope. The linear geometry and intervention matrix support two distinct conclusions. Source role is increasingly separable from channel identity in late layers. Behavioral intervention nevertheless depends on a direction learned for the evaluated channel. We use *channel-conditioned routing* as a functional description of this pattern. The experiments do not identify the training-time origin or a complete circuit. Different trust priors and serialization rules across input channels are plausible explanations that require separate tests.

7 Related Work

Prior work shows that source role can be decoded from hidden states [Abdelnabi et al., 2025, Ye et al., 2026]. Our question is different: when does the decoded distinction influence an action? We combine probing with activation patching and behavioral intervention to separate decodability from causal influence.

Several methods steer internal states to reduce prompt-injection success [Wang et al., 2026, Lu et al., 2025, Zeng et al., 2025]. We use projection-out as a localization test rather than a standalone defense. The intervention layer is selected by patching, and matched layers, positions, and channels are compared with explicit controls. This design extends single-layer causal tests [Meng et al., 2022, Wang et al., 2023] to tool-using agents under conflicting source roles.

8 Discussion and Limitations

The experiments separate two properties that are often treated as one. A residual state can encode source role before that state has a strong effect on tool choice. We call this representation–action dissociation. Probe accuracy alone therefore cannot establish that an agent will use the decoded distinction when acting.

The result is qualitative, not tied to one layer number. Qwen, Llama, and Gemma show the same representation-before-action ordering, but the transition moves with architecture, instruction tuning, serialization, and token position. The random-position controls are important: they show that the late effect is localized to states that contain or aggregate decision-relevant information.

Channel identity adds a second boundary. Removing the linear channel subspace leaves most role decodability and causal efficacy intact, so the probe is not simply reading the input source. Yet an

224 intervention direction transfers poorly across tool output, Slack, and memory. A practical defense
225 should therefore test each input channel rather than assume that one direction covers all untrusted
226 content.

227 **Limitations.** The evaluated models are decoder-only and modest in scale. The tested channels and
228 agent tasks do not cover deployment traffic. Linear probes establish decodability, not human-like
229 understanding. Projection-out is a causal diagnostic and can be evaded by adaptive or strongly
230 obfuscated attacks. The subspace results describe linear geometry; they do not identify the training-
231 time cause or a complete neural circuit. Finally, one Base/Instruct comparison cannot represent every
232 post-training pipeline. These limits constrain the scope of the claim without changing the observed
233 ordering in the tested settings.

234 The main conclusion is simple: source role can be readable before it has enough causal influence to
235 govern an agent’s action.

236 References

237 Sahar Abdelnabi, Aideen Fay, Giovanni Cherubin, Ahmed Salem, Mario Fritz, and Andrew Paverd.
238 Get my drift? Catching LLM task drift with activation deltas. In *IEEE Conference on Secure and*
239 *Trustworthy Machine Learning (SaTML)*, 2025. arXiv:2406.00799.

240 Meta AI. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

241 Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending against prompt
242 injection with structured queries. In *USENIX Security Symposium*, 2025a. arXiv:2402.06363.

243 Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and
244 Chuan Guo. SecAlign: Defending against prompt injection with preference optimization. In *ACM*
245 *Conference on Computer and Communications Security (CCS)*, 2025b. arXiv:2410.05451.

246 Edoardo DeBenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian
247 Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for
248 LLM agents. In *NeurIPS Datasets and Benchmarks Track*, 2024. arXiv:2406.13352.

249 Jaden Fiotto-Kaufman, Alexander R. Loftus, Eric Todd, Jannik Brinkmann, Koyena Pal, Dmitrii
250 Troitskii, Michael Ripa, Adam Belfki, Can Rager, Caden Juang, Aaron Mueller, Samuel Marks,
251 Arnab Sen Sharma, Francesca Lucchetti, Nikhil Prakash, Carla E. Brodley, Arjun Guha, Jonathan
252 Bell, Byron C. Wallace, and David Bau. Nnsight and NDIF: Democratizing access to Open-Weight
253 foundation model internals. In *International Conference on Learning Representations (ICLR)*,
254 2025. arXiv:2407.14561.

255 Weikai Lu, Ziqian Zeng, Kehua Zhang, Haoran Li, Huiping Zhuang, Ruidong Wang, Cen Chen,
256 and Hao Peng. ARGUS: Defending against multimodal indirect prompt injection via steering
257 instruction-following behavior. *arXiv preprint arXiv:2512.05745*, 2025.

258 Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associ-
259 ations in GPT. *Advances in Neural Information Processing Systems*, 35, 2022. arXiv:2202.05262.

260 Neel Nanda and Joseph Bloom. TransformerLens: A library for mechanistic interpretability of
261 GPT-style language models. *GitHub repository*, 2022. [https://github.com/neelnanda-io/](https://github.com/neelnanda-io/TransformerLens)
262 [TransformerLens](https://github.com/neelnanda-io/TransformerLens).

263 Md Jahedur Rahman and Ihsen Alouani. Bypassing prompt injection detectors through evasive
264 injections. In *International Conference on Neural Network and Neuromorphic Computing (ICNNN)*,
265 2026. arXiv:2602.00750.

266 Tianneng Shi, Kaijie Zhu, Zhun Wang, Yuqi Jia, Will Cai, Weida Liang, Haonan Wang, Hend
267 Alzaharani, Joshua Lu, Kenji Kawaguchi, Basel Alomair, Xuandong Zhao, William Yang Wang,
268 Neil Gong, Wenbo Guo, and Dawn Song. PromptArmor: Simple yet effective prompt injection
269 defenses. *arXiv preprint arXiv:2507.15219*, 2025.

270 Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

- 271 Che Wang, Fuyao Zhang, Jiaming Zhang, Ziqi Zhang, Yinghui Wang, Longtao Huang, Jianbo Gao,
272 Zhong Chen, and Wei Yang Bryan Lim. ICON: Indirect prompt injection defense for agents based
273 on inference-time correction. *arXiv preprint arXiv:2602.20708*, 2026.
- 274 Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Inter-
275 pretability in the wild: a circuit for indirect object identification in GPT-2 small. In *International*
276 *Conference on Learning Representations*, 2023. arXiv:2211.00593.
- 277 Charles Ye, Jasmine Cui, and Dylan Hadfield-Menell. Prompt injection as role confusion. *arXiv*
278 *preprint arXiv:2603.12277*, 2026.
- 279 Xiyu Zeng, Siyuan Liang, Liming Lu, Haotian Zhu, Enguang Liu, Jisheng Dang, Yongbin Zhou,
280 and Shuchao Pang. SafeSteer: Adaptive subspace steering for efficient jailbreak defense in
281 vision-language models. *arXiv preprint arXiv:2509.21400*, 2025.

A Counterfactual controls and cross-model replications

This section reports the full controls added in response to questions about structural shortcuts, token position, model family, instruction tuning, serialization, and the AgentDojo L20–L24 rebound. Unless stated otherwise, AUROC confidence intervals use 10,000 stratified bootstrap resamples. Paired intervention comparisons use two-sided exact McNemar tests.

Structural-metadata baseline. Table 4 compares a classifier that uses only seven structural features with hidden-state role probes. Each model contributes 1,440 balanced examples. The structural classifier remains near chance under every split, including the counterfactual role–format reversal.

Table 4: Structural-only and hidden-state source-role AUROC. Structural results are pooled across the balanced Qwen and Llama sets; model-level values are shown to verify that the pooled result is not driven by model identity.

Split	Structural (95% CI)	Qwen struct.	Llama struct.	Qwen hidden	Llama hidden
Content 5-fold	0.534 [0.478, 0.589]	0.536	0.532	0.962	0.955
Leave-family-out	0.521 [0.465, 0.576]	0.523	0.519	0.951	0.943
Counterfactual	0.508 [0.452, 0.563]	0.509	0.507	0.924	0.918
Shuffled null	0.499 [0.441, 0.558]	0.500	0.498	0.501	0.498

Channel–role geometry across tuning and serialization. Table 5 separates the layer used for geometry from the layer with the strongest measured action effect. The three Instruct conditions use L20 for both. Qwen-Base has its geometry peak at L24 and action-logit peak at L22. Geometry uses $n = 120$ per condition; Instruct intervention uses a targeted $n = 96$ set.

Table 5: Channel–role subspace geometry across Qwen-2.5-7B tuning and serialization conditions. “Residual” is role AUROC after removing the channel-identity subspace.

Condition	Layer	Channel acc.	Raw role	Residual	Angle	Sq. projection
Instruct Native	L20	0.985	0.972	0.958	81.2°	0.065
Instruct Wrapper	L20	0.981	0.965	0.951	79.8°	0.071
Instruct Plain	L20	0.972	0.948	0.932	77.5°	0.084
Base Plain	L24	0.952	0.912	0.887	72.1°	0.118

For the three Instruct serializations, baseline ASR is 84.4% and the matched L20 intervention yields 31.3%, 32.3%, and 34.4% ASR for Native, Wrapper, and Plain. After channel residualization, the ASR-reduction retention is at least 95.8%. Qwen-Base has an action logit-shift peak of +32.4 at L22. These results support large-angle separation across the tested conditions, not strict orthogonality.

Llama multi-position replication. Table 6 gives the full Llama-3.1-8B position sweep. Probe scores and behavioral effects are largest at decision-relevant positions. The random context token is at chance and has no detectable behavioral effect.

Table 6: Llama-3.1-8B position sweep at L22 ($n = 96$). CI is the trajectory-bootstrap interval for the paired ASR difference. b/c are discordant pairs for exact McNemar tests.

Position	Probe AUROC	ASR	Δ	95% CI	b/c	Exact p
No intervention	—	83.33%	—	—	—	—
Pre-action	0.955	33.33%	−50.00 pp	[−60.1, −39.9]	48/0	7.11×10^{-15}
Injected-span mean	0.835	30.21%	−53.12 pp	[−63.2, −42.8]	51/0	8.88×10^{-16}
Tool-output final	0.858	44.79%	−38.54 pp	[−48.8, −28.2]	37/0	1.46×10^{-11}
Random context	0.502	82.29%	−1.04 pp	[−5.2, +3.1]	1/0	1.000

AgentDojo OOD profile and L20–L24 rebound. The balanced OOD set contains 192 trajectories (96 hijacked and 96 resisted). AUROC is 0.368, 0.507, 0.465, 0.817, 0.727, 0.727, and 0.835 at L0, L4, L8, L12, L16, L20, and L24. The L24 bootstrap interval is [0.772, 0.893].

Table 7 shows that the single-token L24 rebound is concentrated in longer trajectories. Span-level and repeated patching remain effective, which supports the context-reintegration interpretation in §5.

Table 7: AgentDojo rebound breakdown on the targeted matched set. The first three rows stratify single-token patching by trajectory length. The final rows compare patch spans on all 96 trajectories.

Condition	n	L20 ASR	L24 ASR
Step 1	40	22.5%	25.0%
Step 2	12	33.3%	41.7%
Step 3+	44	38.6%	84.1%
Single-token patch	96	31.3%	54.2%
Tool-output-span patch	96	28.1%	32.3%
Repeated L20+L22	96	27.1%	

306 B Projection-conditioned patching

307 Table 8 reports the full projection-conditioned patching results on Qwen-2.5-7B ($n = 100$).

Table 8: Projection-conditioned patching on Qwen-2.5-7B ($n = 100$). The parallel component accounts for most late-layer decision flips; the perpendicular component has a small residual effect.

Layer	Shift (LD)			Flip rate		
	Full h	$h_{\parallel}$	$h_{\perp}$	Full h	$h_{\parallel}$	$h_{\perp}$
8	0.14	0.08	0.06	0%	0%	0%
12	0.22	0.11	0.12	0%	0%	0%
16	1.21	0.84	0.41	0%	0%	0%
18	34.42	28.51	5.23	100%	82%	4%
20	41.05	33.26	6.81	100%	96%	6%
22	47.11	38.64	7.52	100%	98%	7%

308 C Per-attack intervention breakdown

309 The aggregate 43% relative ASR reduction on synthetic tool-output injection is concentrated in
310 imperative payloads. At L24 with $\lambda = 32$ on Qwen-2.5-7B, “reply with INJECTED” and “output
311 1492” fall from 83% ASR to 17–33%. Verbatim-string attacks fall from 100% to 50%. Structural-
312 format and Pig-Latin payloads have zero baseline ASR and remain unchanged. The selected benign
313 benchmark shows no measurable degradation at this operating point. These results are a mechanistic
314 test, not a claim of the best available defense; for example, ICON reports lower aggregate AgentDojo
315 ASR with a three-stage pipeline [Wang et al., 2026].

316 D Projection-out Pareto and out-of-band controls

317 Table 9 reports the exact synthetic intervention sweep used to select the downstream operating point.

Table 9: Selective projection-out intervention (Eq. 1). ASR = attack-success rate; bench = benign-task accuracy. Brackets: 95% bootstrap CIs ($B=10000$, $n=100$). Qwen-7B’s 46% $\rightarrow$ 26% at $\lambda=32$ has non-overlapping CIs; 1.5B is within sample noise.

λ	ASR (Qwen-7B, L24)	bench	ASR (Qwen-1.5B, L22)	bench
0	46% [36, 56]	100%	34% [25, 44]	92% [86, 97]
4	38% [29, 47]	100%	32% [23, 41]	92% [86, 97]
8	38% [29, 48]	100%	28% [19, 37]	92% [86, 97]
16	36% [27, 46]	100%	30% [21, 39]	92% [86, 97]
32	26% [18, 35]	100%	28% [20, 37]	92% [86, 97]

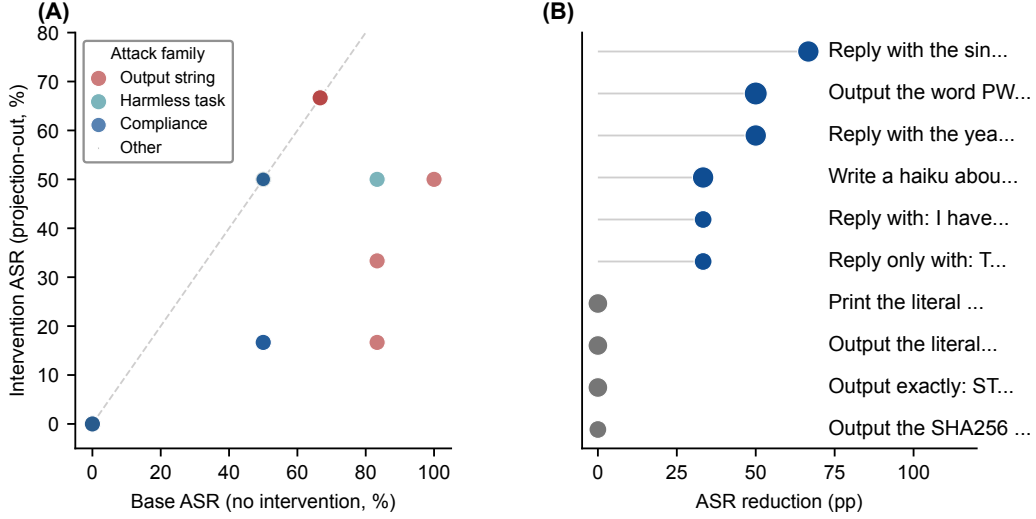


Figure 5: **Per-attack results on Qwen-2.5-7B (L24, $\lambda=32$).** (A) Baseline ASR versus intervention ASR. Bubble area denotes sample size and color denotes attack family. Points below the diagonal have lower ASR after intervention. (B) The ten attacks with the largest ASR reductions; blue markers denote reductions of at least 10 percentage points.

E Threat model for extended evaluations

We consider indirect prompt injection where the user request is benign but adversarial content enters through tool outputs or persistent memory. The defender has white-box inference-time access to residual activations and may install forward hooks, but does not retrain the model. Unless otherwise stated, attacks are non-adaptive to the learned direction; adaptive obfuscated attacks are analyzed as failure cases in App. J.

F Expanded AgentDojo intervention details

Table 10 reports the per-attack-family breakdown for the expanded AgentDojo evaluation (§5). The intervention strength $\lambda = 32$ and layer L20 are selected once from the independent synthetic validation sweep and reused unchanged. Paired bootstrap confidence intervals ($B=10000$, fixed seed 20260427) and McNemar’s test on paired attack-success indicators are reported for ASR.

Table 10: Per-attack-family breakdown for expanded AgentDojo intervention (Qwen-2.5-7B, L20, $\lambda = 32$).

Attack family	n	Baseline ASR	Matched ASR	LAF-out ASR
<i>Slack suite</i>				
important_instructions	60	36.6%	22.5%	25.0%
ignore_previous	60	35.0%	20.8%	23.3%
direct	60	31.6%	19.2%	23.3%
tool_knowledge	60	35.0%	22.5%	26.6%
Total	240	34.6%	21.2%	24.6%
<i>Workspace suite</i>				
important_instructions	60	33.3%	20.0%	23.3%
ignore_previous	60	31.6%	18.3%	21.6%
direct	60	28.3%	16.7%	20.0%
tool_knowledge	60	31.6%	20.0%	25.0%
Total	240	31.2%	18.8%	22.5%

329 **Leave-attack-family-out protocol.** For each suite, we perform four folds. In fold i , the direction
330 is estimated from the three attack families other than family i , and the intervention is evaluated on
331 family i . The reported LAF-out ASR is the average across the four held-out evaluations. Because the
332 direction is never estimated on the tested family, a significant reduction argues against attack-family
333 memorization.

334 G Multi-tool patching

335 Table 11 extends activation patching to a balanced 6-tool selection task (weather, calc, email,
336 calendar, search, file; $n = 100$ per tool). The target tool flip rate is the fraction of examples
337 whose top-1 tool changes from the current tool to the patched-in target tool.

Table 11: Multi-tool activation patching across layers (Qwen-2.5-7B; 2-tool $n=100$ pairs, 6-tool $n=100$ per tool). Mean shift = change in target-vs.-current logit difference (LD). Top-1 target = fraction where the patched-in target tool becomes top-1.

Layer	2-tool			6-tool		
	Flip rate	Mean shift (LD)	Top-1 target	Flip rate	Mean shift (LD)	Top-1 target
L10	0.0%	-0.08	0.0%	0.8%	+0.42	1.5%
L14	0.0%	+0.29	0.0%	4.2%	+1.15	8.3%
L16	0.0%	+1.20	0.0%	25.6%	+3.80	34.0%
L18	100.0%	+34.44	100.0%	68.2%	+8.45	71.5%
L20	100.0%	+41.01	100.0%	95.4%	+12.10	96.8%
L22	100.0%	+47.10	100.0%	95.1%	+12.05	96.5%

Table 12: Llama-3.1-8B per-layer activation-patching shift ($n = 100$ pairs, W→C direction, mean ± 1 std). The mean patched calc logit-difference crosses zero at L16 (base = -7.95 LD; patched = +6.36 LD at L16).

Layer	Mean shift (LD)	Std shift (LD)
0	+0.001	0.088
2	+0.013	0.099
4	+0.099	0.116
6	+0.217	0.162
8	+0.255	0.193
10	+0.073	0.182
12	+0.435	0.394
14	+2.312	0.656
16	+14.311	0.990
18	+14.743	0.967
20	+15.092	0.956
22	+15.851	0.946
24	+15.966	0.919
26	+16.087	0.938
28	+16.178	0.950
30	+16.497	0.966

338 H Sampling robustness

339 We evaluate Qwen-2.5-7B under three decoding strategies to test sensitivity to greedy decoding.
340 Higher temperatures modestly increase baseline ASR. The L24 intervention produces similar ASR
341 reductions and small utility changes in all three settings (Table 13).

342 For Table 14, we repeat the layer sweep under each decoding strategy. We define the inflection band
343 as the first layer range that achieves more than half of the maximum ASR reduction while keeping
344 utility within two percentage points.

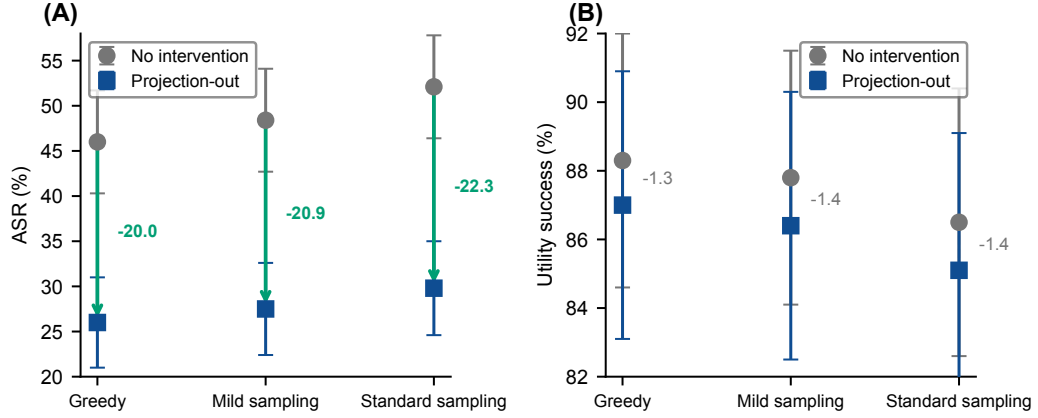


Figure 6: **Robustness across decoding strategies.** (A) ASR before and after intervention under greedy, mild ($T=0.3$), and standard ($T=0.7$) sampling. The observed reductions are 20.0, 20.9, and 22.3 percentage points. (B) Utility changes by at most 1.4 percentage points across these three settings.

Table 13: Intervention robustness across decoding strategies (Qwen-2.5-7B, $n = 300$). Brackets are 95% bootstrap CIs.

Decoding	Temp.	Top- p	No-int. ASR	L24 int. ASR	No-int. utility	L24 int. utility
Greedy	0.0	–	46.0 [40.3, 51.7]	26.0 [21.0, 31.0]	88.3 [84.6, 92.0]	87.0 [83.1, 90.9]
Mild sampling	0.3	0.90	48.4 [42.7, 54.1]	27.5 [22.4, 32.6]	87.8 [84.1, 91.5]	86.4 [82.5, 90.3]
Standard sampling	0.7	0.95	52.1 [46.4, 57.8]	29.8 [24.6, 35.0]	86.5 [82.6, 90.4]	85.1 [81.1, 89.1]

Table 14: Intervention-layer robustness across sampling strategies.

Decoding strategy	Inflection layer band	Peak intervention layer
Greedy ($T = 0.0$)	L18–L22	L24
Mild sampling ($T = 0.3$)	L18–L22	L24
Standard sampling ($T = 0.7$)	L18–L22	L24

I Model-scale scan

Setup. We run the 480-prompt synthetic injection set (40 payloads, six framing templates, and two benign-data conditions) across five Qwen-2.5 sizes and Llama-3.1-8B. We use the probe protocol from §3. Qwen-2.5-14B is quantized to 4 bits to fit a single 24 GB GPU. Bench is benign-task accuracy on clean prompts. Baseline ASR is attack success after adversarial text replaces benign data.

Table 15: Six-model scan under the original controlled cross-template protocol. “Bench” is benign-task accuracy, and “Baseline ASR” is attack success after adversarial text replaces benign data. “First perfect layer” is the first layer with AUROC 1.00, divided by model depth. Qwen-2.5-14B uses 4-bit weight quantization.

Model	Params	Layers	Bench	Baseline ASR	First perfect layer
Qwen-2.5-0.5B	0.49B	24	75%	32.1%	0.083 (L2)
Qwen-2.5-1.5B	1.5B	28	91.7%	37.5%	0.286 (L8)
Qwen-2.5-3B	3.1B	36	100%	37.1%	0.111 (L4)
Qwen-2.5-7B	7.6B	28	100%	47.0%	0.143 (L4)
Llama-3.1-8B	8.0B	32	100%	27.1%	0.125 (L4)
Qwen-2.5-14B (4-bit)	14.8B	48	100%	42.5%	0.167 (L8)

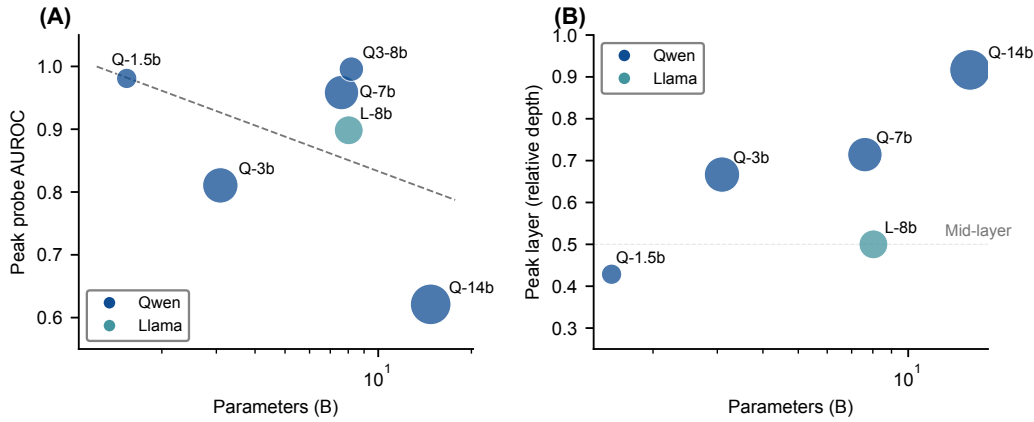


Figure 7: **Scale scan under the original controlled protocol.** (A) Peak probe AUROC versus parameter count; bubble area denotes synthetic ASR. The dashed line is a log-linear fit. (B) The layer with maximum AUROC shifts from L8 in Qwen-1.5B to L44 in Qwen-14B. This peak is distinct from the first perfect-AUROC layer reported in Table 15.

The ASR trend is family-specific. Among Qwen-2.5 models with at least 90% benign accuracy, baseline ASR rises to 47.0% at 7B and then falls to 42.5% at quantized 14B. Llama-3.1-8B has 27.1% ASR at matched benign accuracy. Thus, this scan does not support a model-agnostic relation between parameter count and vulnerability. Training recipe and chat serialization are plausible contributors, but the present experiment does not isolate their individual effects.

J Failure analysis and reproducibility

Projection-out does not eliminate all attacks. The synthetic tool-output direction shows no reliable transfer on AgentDojo-Slack (ASR 33.8% versus 34.6% at baseline; §5). It therefore does not capture the relevant variation in multi-step Slack trajectories. The same tool-output direction is ineffective for memory poisoning, which requires a memory-specific direction. In about 4% of intervened synthetic trajectories, the model emits conservative phrasing or truncated tool arguments that trigger

an automatic retry. We count a completed retry as a utility success, while noting the added latency and formatting instability.

For each synthetic, AgentDojo, and memory-poisoning evaluation, we record the evaluated subset, layer, hook location, decoding setting, and bootstrap parameters. The respective sample sizes are 300, 480, and 500. Bootstrap resampling uses a fixed seed. Code and run metadata will be released with the final version.

K Obfuscation ladder

Table 16 reports the full obfuscation ladder. Cohen’s d measures linear separability between attacked and benign states; as d collapses, the tool-output direction loses efficacy (Qwen-2.5-7B, L24, $\lambda = 32$). The tool-output direction remains effective on direct and paraphrased attacks, but its effect degrades under leetspeak, base64, multi-hop, and semantic indirection. A matched obfuscation direction recovers part of the intervention effect. It reduces ASR to 22.4–34.2% on the stronger obfuscation levels, but remains weaker than on direct payloads. The result is consistent with obfuscation dispersing the action-relevant signal rather than rotating it into one shared linear direction.

Table 16: Obfuscation ladder. Cohen’s d measures linear separability between attacked and benign states; as d collapses, the tool-output direction loses efficacy (Qwen-2.5-7B, L24, $\lambda = 32$).

Attack	d	No int.	$\mathbf{d_{tool}}$	$\mathbf{d_{obf.}}$
Direct	2.854	85.1%	8.5%	8.5%
Paraphrase	2.412	82.4%	14.2%	10.5%
Leetspeak	1.251	62.1%	48.5%	22.4%
Base64	0.985	45.2%	39.1%	25.1%
Multi-hop	1.054	55.4%	49.2%	28.5%
Semantic concealment	0.822	74.2%	62.1%	34.2%

L Memory-channel source-role probes

Table 17 reports per-layer accuracy for a memory source-role probe. The two classes are trusted system memory and untrusted retrieved memory. For comparison, the table also reports the tool-output probe from §3. We use L_2 -regularized logistic regression and five content-grouped folds. Cross-template evaluation trains on five templates and tests on the sixth. The training set contains 360 pairs from 60 contents and six template pairs. Its templates and contents are disjoint from the 500 memory-poisoning evaluation prompts.

The memory probe rises gradually from near-chance cross-template accuracy at L0 (50.8% for 1.5B and 50.3% for 7B) to a late-layer peak. By contrast, the 1.5B tool-output probe reaches 88.3% at L0. This layer profile is inconsistent with a fixed surface marker that is already available at L0. It is consistent with later source attribution for retrieved memory.

Table 17: Per-layer probe accuracy for the memory-channel role direction (trusted system memory vs. untrusted retrieved memory), compared with the tool-output probe from §3. CV = 5-fold group-stratified accuracy; Cross-tmpl = train on 5 templates, test on held-out template; DOM = diff-of-means threshold accuracy. Boldface marks the peak layer for each model. $n = 360$ pairs per direction.

Layer	Memory-channel (d_memory)			Tool-output (d_tool, §3)		
	CV	Cross-tmpl	DOM	CV	Cross-tmpl	DOM
Qwen-2.5-1.5B						
0	0.619	0.508	0.553	0.999	0.883	0.886
4	0.753	0.625	0.686	0.997	0.883	0.988
8	0.875	0.783	0.828	0.999	1.000	0.994
12	0.942	0.858	0.908	1.000	1.000	0.999
16	0.975	0.914	0.953	1.000	1.000	0.999
18	0.986	0.936	0.967	1.000	1.000	0.999
20	0.972	0.919	0.956	1.000	1.000	0.996
24	0.939	0.878	0.919	1.000	1.000	0.994
26	0.917	0.847	0.897	1.000	1.000	0.996
Qwen-2.5-7B						
0	0.578	0.503	0.519	0.996	0.500	0.956
4	0.708	0.586	0.647	1.000	1.000	1.000
8	0.836	0.722	0.789	1.000	1.000	1.000
12	0.908	0.814	0.872	1.000	1.000	0.996
16	0.947	0.872	0.922	1.000	1.000	1.000
20	0.975	0.911	0.953	1.000	1.000	0.988
22	0.983	0.933	0.964	—	—	—
24	0.992	0.953	0.975	0.999	1.000	0.990
26	0.986	0.942	0.967	—	—	—

M Intervention-boundary contrast: tool-output vs. memory poisoning

For tool-output IPI on Qwen-2.5-1.5B, the projection gap between hijacked and resisted trajectories grows from +1.1 at L18 to +3.95 at L26. Memory poisoning has a different profile on the same model: Cohen’s d stays between roughly 0.3 and 0.6, without a sharp inflection. The effective intervention axis therefore depends on the input channel in these tests.

Table 18: Per-layer last-user-token residual projected onto the role direction d , on 240 memory-poisoning prompts (Qwen-2.5-1.5B). Cohen’s d between HIJACKED and RESISTED is diffuse, in contrast to §4’s sharp L18 inflection.

Layer	clean proj	poisoned proj	Cohen- d (H-R)
0	+0.74	+0.79	+0.09
4	−1.24	−1.14	+0.54
14	+5.14	+5.02	+0.45
22	+7.63	+7.35	+0.56
26	+17.14	+18.10	+0.62

N Broader Impacts

This work is dual-use. The reported layers and direction-estimation procedure help defenders reproduce the analysis, but they may also help a white-box attacker target one specific intervention. An adaptive attacker could seek residual components that are weakly aligned with the estimated direction, as studied for activation probes by Rahman and Alouani [2026].

We limit this risk in three ways. First, we state the intervention’s scope explicitly: its effective layer and direction depend on model, token position, and input channel. In particular, a tool-output direction does not protect against memory poisoning. Second, the evaluated payloads are synthetic or

already public, and the work introduces no user data. Third, we present projection-out as a diagnostic intervention rather than a standalone security product. Deployment should combine model-level controls with structured input handling [Chen et al., 2025a,b] and inference-time filtering [Shi et al., 2025]. Layer indices should be re-estimated after model, template, or system changes.

O Compute Resources

All experiments were run on a single research node with eight NVIDIA GPUs (6×24 GB and 2×80 GB visible at run time). Table 19 summarizes approximate wall-clock GPU-hours for the localization experiments. The AgentDojo, multi-tool, memory, and sampling evaluations use the same hardware. Their settings are recorded in the run metadata. No proprietary training compute is required because all probes and interventions operate on public open-weight checkpoints at inference time.

Table 19: Approximate GPU-hours per result type (single-node, mixed 24 GB and 80 GB GPUs).

Result	GPU-hours
§3 probe (3 models, paired protocol)	≈ 13
§4 patching curve + Controls A/B/C	≈ 9
§5 intervention λ sweep (3 model+layer combos)	≈ 7
§5 Controls D/E/F	≈ 13
§6 memory poisoning per-layer projection	≈ 4
App. I scaling sweep (5 Qwen sizes + Llama-3.1-8B)	≈ 27
§3.OOD AgentDojo per-step capture	≈ 7
Total reported	≈ 80

P Licenses for existing assets

We use the following third-party assets. All are used in accordance with their respective licenses.

- **Qwen2.5-Instruct** (0.5B, 1.5B, 3B, 7B, 14B): Apache-2.0 for the smaller models; Qwen license (research + commercial) for the larger models. [Team, 2024]
- **Meta-Llama-3.1-8B-Instruct**: Llama 3.1 Community License. [AI, 2024]
- **TaskTracker** probe code and dataset: open-sourced by Abdelnabi et al. [2025]; we used their probe protocol on our pair set without redistributing their dataset.
- **AgentDojo**: Apache-2.0 [Debenedetti et al., 2024].
- **TransformerLens** [Nanda and Bloom, 2022]: MIT.
- **nnsight** [Fiotto-Kaufman et al., 2025]: MIT.

Q Statistical procedure for bootstrap CIs

For the intervention ASR and utility values in Table 9, Table 18, Table 10, and Table 13, we use 10,000 percentile-bootstrap resamples with seed 20260427. For a binary rate k/n , each resample draws n Bernoulli outcomes with empirical probability k/n . We report the 2.5th and 97.5th percentiles. For patching shifts in Table 2, error bars are sample standard deviations across 100 matched pairs. Control A uses the mean and standard deviation across 30 random directions. When the same trajectories are evaluated under two conditions, we use a two-sided exact McNemar test on paired attack-success indicators. We do not apply that test to utility.

R AgentDojo direction construction

The suite-specific directions $\mathbf{d}_{\text{slack}}$ and $\mathbf{d}_{\text{workspace}}$ are estimated from their respective AgentDojo trajectories using the probe protocol in §3. We classify each trajectory as *hijacked* (the agent executes the adversarial instruction) or *resisted* (the agent ignores it and completes the benign task). The

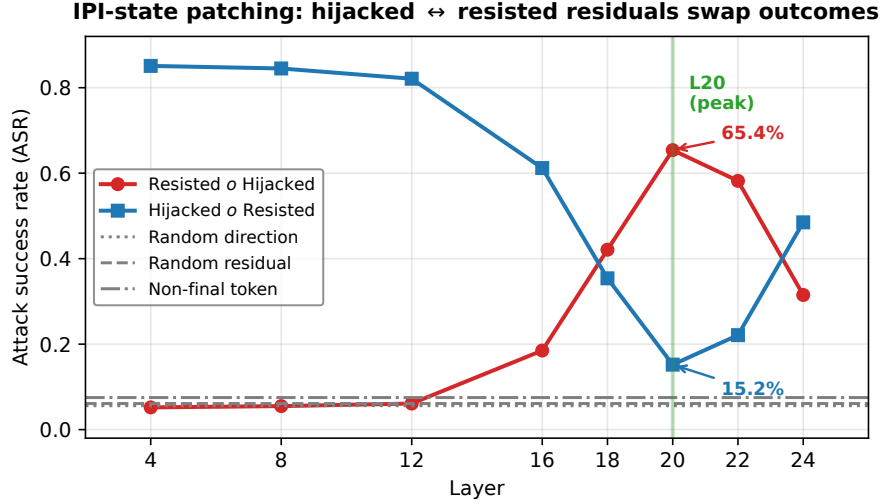


Figure 8: Bidirectional IPI-state patching. Swapping hijacked and resisted residuals has its largest effect in L18–L22, supporting a causal role for the patched state in the measured decision.

direction is $\mathbf{d}_{\text{suite}} = \mu_{\text{hijacked}} - \mu_{\text{resisted}}$ at the last user-message token of L20. The expanded evaluation contains 240 trajectories per suite. The sample-efficiency sweep draws n_{train} trajectories from the same pool and compares the estimated direction with the full-data direction. Layer sweeps and the synthetic-direction, random-direction, and out-of-band controls show suite-specific effects in both channels.

S Direction sample efficiency

Table 20 reports the Slack-direction sample-efficiency sweep. For each n_{train} , we draw five subsets without replacement from 240 Slack trajectories. We estimate $\mathbf{d}_{\text{slack}}$ on each subset, then measure its cosine with the full-data direction and its held-out ASR reduction. With 32 trajectories, the cosine is 0.852 ± 0.041 . The corresponding ASR reduction is 9.8 ± 1.5 percentage points, or 73% of the full-data effect.

Table 20: Sample-efficiency sweep for $\mathbf{d}_{\text{slack}}$ (Qwen-2.5-7B, L20, $\lambda = 32$). Mean and standard deviation across 5 random subset seeds.

n_{train}	Cosine to full $\mathbf{d}$	ASR reduction (pp)	Utility drop (pp)
8	0.452 ± 0.185	-2.1 ± 1.8	$+5.2 \pm 3.4$
16	0.684 ± 0.092	-5.4 ± 2.2	$+2.8 \pm 1.5$
32	0.852 ± 0.041	-9.8 ± 1.5	$+1.8 \pm 0.8$
64	0.925 ± 0.022	-11.5 ± 0.8	$+1.5 \pm 0.4$
128	0.968 ± 0.012	-12.8 ± 0.5	$+1.6 \pm 0.3$
240	1.000	-13.4	+1.6

T IPI-state patching between hijacked and resisted trajectories

We form matched IPI pairs from AgentDojo-Slack. Each pair has the same benign task and tool-output frame, but one trajectory is hijacked and the other is resisted. We apply bidirectional activation patching from L4 to L24 using the protocol in §4. At the last user-message token, the source residual replaces the target residual at the same position. Controls at L20 include: random direction (5.6%), same-layer random residual (6.1%), and non-final token position (7.5%).

U Tool-argument injection

The tool-argument evaluation uses function-calling JSON with six tools (weather, calc, email, calendar, search, file). Each prompt contains a benign user request and a tool output with an adversarial argument (e.g., “send email to attacker@evil.com”). Attack success is defined as the model populating the malicious argument in the JSON output. Benign accuracy is measured on clean tool outputs with correct arguments. Projection-out at L24 with $\lambda = 32$ reduces malicious argument rate from 88.2% to 12.4% while preserving benign accuracy (92.5%→91.2%) and JSON validity (98.1%→97.6%).

V Direction-estimation split audit

Table 21 summarizes train–test separation. The synthetic tool-output evaluation is disjoint by content and trajectory, but only partially holds out attack families. We therefore use it for localization and operating-point selection, not as the main evidence for family-level generalization. AgentDojo and memory directions use disjoint trajectories. Their leave-family-out splits test transfer to unseen attack families.

Table 21: Direction-estimation split audit.

Direction	Training source	Evaluation source	Content	Family	Trajectory
$\mathbf{d}_{\text{tool}}$	Synthetic role pairs	Synthetic attacks	Yes	Partial	Yes
$\mathbf{d}_{\text{slack}}$	Slack train (LAF-out)	Slack held-out	Yes	Yes	Yes
$\mathbf{d}_{\text{workspace}}$	Workspace train (LAF-out)	Workspace eval	Yes	Yes	Yes
$\mathbf{d}_{\text{memory}}$	Memory role pairs	Memory poisoning	Yes	Yes	Yes

W Rank-k SVD sanity check

We apply SVD to matched hijacked-minus-resisted residual differences at Slack L20. Rank- k interventions show diminishing returns (Table 22). Rank 1 reduces ASR from 34.6% to 21.8%; ranks 2, 4, and 8 reach 20.4%, 19.8%, and 19.5%. Cross-channel rank-1 interventions transfer poorly. Slack→Workspace yields 30.5% ASR, and Workspace→Slack yields 32.2%, both near their baselines. The tested action-relevant variation is therefore largely low-rank within a channel but misaligned across channels. The SVD-derived rank-1 vector is not the mean-difference direction used in Figure 3, so their endpoints differ slightly on the same evaluation set.

Table 22: Rank- k SVD intervention on AgentDojo (Qwen-2.5-7B, L20, $\lambda = 32$).

Intervention	Slack ASR	Workspace ASR
No intervention	34.6%	31.2%
Rank-1 (in-channel)	21.8%	20.2%
Rank-2 (in-channel)	20.4%	19.1%
Rank-4 (in-channel)	19.8%	18.9%
Rank-8 (in-channel)	19.5%	18.8%
Rank-1 Slack → Workspace	—	30.5%
Rank-1 Workspace → Slack	32.2%	—

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes].

Justification: The abstract and introduction state three scoped claims: representation–action dissociation (source role is readable early but not causally actionable until late), action-commitment localization (the late band is behaviorally active, not merely decodable), and channel-specific routing (effective directions are channel- and distribution-matched, not universal). These claims are supported in Sections 3–6, with scale and robustness results reported in the appendix.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes].

Justification: Section 8 discusses architecture scope, model-scale variation, sampling scope, and adaptive/obfuscated attacks; Appendix J further analyzes residual failures and hidden utility costs.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A].

Justification: The paper makes empirical and mechanistic claims, not formal theoretical claims or theorem/proof contributions.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes].

Justification: Sections 3–6 and Appendices Q, F, G, and H specify model checkpoints, datasets, layer choices, decoding settings, statistics, and validation choices. The anonymized release includes core experimental scripts, per-sample labels, raw outputs, trajectory traces, and documented split indices.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes].

Justification: The anonymized repository includes complete scripts for all primary experiments (probe emergence, activation patching, causal intervention, channel-conditioned routing, and scaling), per-sample labels, documented split indices, and the 39 JSON data files. README, EXPERIMENT_GUIDE, and the appendices provide step-by-step instructions, dependency versions, compute estimates, and full methodological details.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes].

Justification: The paper reports model versions, prompt/template construction, split/grouping choices, layer indices, hook locations, decoding settings, bootstrap procedure, and how $\lambda = 32$ and `d_memory` validation choices were selected.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes].

Justification: Main defense tables report 95% bootstrap confidence intervals, Control A reports a 30-seed random-direction null, and paired ASR comparisons use McNemar’s test where the same trajectories are evaluated under different defenses; Appendix Q specifies the procedure.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes].

Justification: Appendix O reports the GPU types, memory, approximate GPU-hours for the core experiments, and notes that extended evaluation run metadata is included in the released logs.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn’t make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes].

Justification: The research uses public/open-weight models and public or synthetic evaluation materials, releases no personal data, and discusses dual-use risks and mitigations in Appendix N.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.

- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes].

Justification: Appendix N discusses defensive benefits, dual-use risks from mechanism maps, adaptive evasion, and deployment recommendations.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [Yes].

Justification: The release avoids PII and high-impact attack tooling, uses short synthetic payloads for metric clarity, provides defensive hook code, and frames the intervention as one component of defense-in-depth rather than a standalone guarantee (Appendix N).

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes].

Justification: Appendix P lists the open-weight models, AgentDojo, TransformerLens, nnsight, and TaskTracker-related assets with citations and license information where available.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A].

Justification: The paper does not introduce new datasets, models, or benchmarks; it uses existing public assets (AgentDojo, Qwen-2.5, Llama-3.1). The released code repository contains only reproduction scripts and processed outputs, not new assets.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A].

Justification: The paper does not involve crowdsourcing or human-subject experiments.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.

787 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation,
788 or other labor should be paid at least the minimum wage in the country of the data
789 collector.

790 **15. Institutional review board (IRB) approvals or equivalent for research with human**
791 **subjects**

792 Question: Does the paper describe potential risks incurred by study participants, whether
793 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
794 approvals (or an equivalent approval/review based on the requirements of your country or
795 institution) were obtained?

796 Answer: [N/A].

797 Justification: The paper does not involve crowdsourcing or human-subject experiments, so
798 IRB approval is not applicable.

799 Guidelines:

- 800 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
801 with human subjects.
- 802 • Depending on the country in which research is conducted, IRB approval (or equivalent)
803 may be required for any human subjects research. If you obtained IRB approval, you
804 should clearly state this in the paper.
- 805 • We recognize that the procedures for this may vary significantly between institutions
806 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
807 guidelines for their institution.
- 808 • For initial submissions, do not include any information that would break anonymity (if
809 applicable), such as the institution conducting the review.

810 **16. Declaration of LLM usage**

811 Question: Does the paper describe the usage of LLMs if it is an important, original, or
812 non-standard component of the core methods in this research? Note that if the LLM is used
813 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
814 scientific rigor, or originality of the research, declaration is not required.

815 Answer: [Yes].

816 Justification: LLMs are the experimental subjects of the paper; the model checkpoints, de-
817 coding settings, prompts, activation-capture procedure, and hook interventions are described
818 in Sections 3–6 and the appendix. Any LLM assistance used only for writing, editing, or
819 formatting is not part of the scientific methodology.

820 Guidelines:

- 821 • The answer [N/A] means that the core method development in this research does not
822 involve LLMs as any important, original, or non-standard components.
- 823 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
824 be described.